



Security Assessment Report



Silo Dual Oracle

June 2026

Prepared for Silo Finance

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	8
Severity Matrix.....	8
Detailed Findings.....	9
Low Severity Issues.....	10
L-01: Re-pricing a pending override restarts the timelock, so it may never activate for a volatile asset.....	10
L-02: renounceOwnership() implicitly makes DualOracle pause and override states permanent.....	11
L-03: Pausing can consume the manual-override timelock while the market is frozen.....	13
Informational Severity Issues.....	15
I-01: DualOracleFactory allows nested DualOracle deployments.....	15
I-02: No minimum timelock, or stale interface doc.....	17
I-03: DualOracle's initialize does not verify that the wrapped oracle can quote.....	18
I-04: DualOracleFactory can execute arbitrary calls during underlying oracle deployment.....	19
Design Recommendations.....	20
1. Split emergency and pricing authorities.....	20
2. Consider rate-limited or TWAP-style manual price movement.....	20
3. Make the return to primary pricing explicit.....	21
4. Consider an emergency price-limit mode.....	21
Disclaimer.....	22
About Certora.....	22

Project Summary

Project Scope

Project Name	Repository Link	Initial Commit Hash	Final Commit Hash	Platform
Silo Dual Oracle	https://github.com/silo-finance/silo-contracts-v3	20ea558	d40504e	EVM / Solidity

Project Overview

This document describes the security review of **Silo Dual Oracle**. The work was undertaken from **June 10th, 2026** to **June 12th, 2026**.

The following contract list is included in our scope:

silo-oracles/contracts/dualOracle/DualOracle.sol

silo-oracles/contracts/interfaces/IDualOracle.sol

silo-oracles/contracts/dualOracle/DualOracleFactory.sol

silo-oracles/contracts/interfaces/IDualOracleFactory.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Silo Finance is a lending protocol built around isolated markets, where each market has its own assets, risk parameters, and oracle setup. The core protocol uses oracle prices to calculate account solvency, borrowing power, withdrawals, and liquidations, so oracle behavior directly affects whether a market can operate normally.

The Dual Oracle is a wrapper around an existing Silo oracle. Under normal conditions it forwards price requests to the underlying oracle, but it also gives the owner two emergency controls: pausing price reads entirely, and setting a bounded manual price that becomes active after a configured timelock. The Dual Oracle stores the underlying oracle, the quote token, the base token, price bounds, timelock, and manual override state. The factory deploys Dual Oracle instances deterministically, either by wrapping an already deployed oracle or by first calling another oracle factory to deploy the underlying oracle and then wrapping it.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The main trust boundary is between Silo core, which consumes oracle prices, and the Dual Oracle wrapper, which can either forward to the primary oracle or replace it with owner-controlled behavior. The sections below summarize the main assets, actors, assumptions, and non-exhaustive attack vectors considered during the review, with focus on wrapped-oracle correctness, market liveness, owner controls, factory deployment flows, and timelock semantics.

Assets

- **Silo market positions** – collateral, debt, and user solvency state.
- **Oracle quote outputs** – prices used for LTV and liquidation checks.
- **Market liveness** – borrowing, withdrawals, liquidations, and solvency checks.
- **Manual override state** – `manualPrice`, `overrideValidAt`, and price bounds.
- **Pause state** – global stop switch for `quote` and `beforeQuote`.
- **Oracle configuration** – wrapped oracle, base token, quote token, and decimals.
- **Ownership authority** – control over `pause`, `unpause`, and manual prices.
- **Deterministic deployments** – factory salts, clone addresses, and `createdInFactory`.

Actors

- **Dual Oracle owner** – controls pause and manual override.
- **Silo users** – depositors and borrowers affected by oracle prices.
- **Liquidators** – rely on fresh and correct solvency quotes.
- **Silo market deployer** – configures markets and oracle deployments.
- **Dual Oracle factory caller** – creates wrappers and passes init data.

Trust Assumptions

- **Wrapped oracle is Silo-compatible** – exposes correct base and quote behavior.

- **Owner is trusted** – pause and manual prices are not abused.
- **Manual bounds are correct** – configured around the intended quote units.
- **Timelock semantics are understood** – delay matches the intended user protection.
- **Pause semantics are understood** – paused markets may be unavailable.
- **Deterministic address assumptions hold** – salts and caller context are correct.

Attack Vectors

- **Consuming or bypassing the manual-override delay** through zero timelock or pause time counting toward `overrideValidAt`.
- **Keeping a pending manual override from activating** by repeatedly repricing before the timelock expires.
- **Permanently freezing or pinning oracle behavior** by renouncing ownership while paused or override-active.
- **Creating layered oracle behavior** by wrapping a `DualOracle` inside another `DualOracle`.
- **Deploying a dead-on-arrival wrapper around an oracle** that exposes metadata but cannot return a valid quote.
- **Misconfiguring manual prices or bounds** because quote units are confused with fixed 18-decimal units.
- **Triggering market-wide reverts through beforeQuote**, since its failures propagate into Silo core flows.
- **Breaking underlying oracle deployment** through low-level call return data, revert, or gas edge cases.
- **Racing deterministic deployment flows** to attempt ownership or address hijacking.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	3	3	2
Informational	4	4	1
Total	7	7	3

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Re-pricing a pending override restarts the timelock, so it may never activate for a volatile asset	Low	Fixed
L-02	renounceOwnership() implicitly makes DualOracle pause and override states permanent	Low	Acknowledged
L-03	Pausing can consume the manual-override timelock while the market is frozen	Low	Fixed
I-01	DualOracleFactory allows nested DualOracle deployments	Informational	Acknowledged
I-02	No minimum timelock, or stale interface doc	Informational	Fixed
I-03	DualOracle's initialize does not verify that the wrapped oracle can quote	Informational	Acknowledged
I-04	DualOracleFactory can execute arbitrary calls during underlying oracle deployment	Informational	Acknowledged

Low Severity Issues

L-01: Re-pricing a pending override restarts the timelock, so it may never activate for a volatile asset

Severity: Low	Impact: Low	Likelihood: Medium
Files: silo-oracles/contracts/dualOracle/DualOracle.sol	Status: Fixed	

Description:

While a manual-price override is pending (`!isOverrideActive()`), any re-set of the price restarts the activation timelock from the current block. For a volatile asset whose fair value drifts during the timelock window, an operator who re-prices to track the move keeps pushing the activation timestamp forward, so the override can be deferred indefinitely and never engage. This degrades the availability of the override as a safety mechanism precisely under the volatile conditions it is meant to handle.

Recommendations:

Allow a pending price to be adjusted within a tight band of the original pending value without resetting the clock.

Customer Response:

Fixed in [d40504e](#).

Fix Review:

Fixed. Re-pricing a pending override no longer resets the timelock. One minor residual tradeoff is that a price setter can first queue an acceptable-looking price, let the timelock nearly elapse, then replace it with a materially different value right before activation, causing that later value to become active immediately at the original `overrideValidAt` without ever using the initially queued price.

L-02: `renounceOwnership()` implicitly makes `DualOracle` pause and override states permanent

Severity: Low	Impact: Low	Likelihood: Low
Files: silo-oracles/contracts/dualOracle/DualOracle.sol	Status: Acknowledged	

Description:

`DualOracle` inherits `renounceOwnership()` from `Ownable2StepUpgradeable` and does not override it. As a result, the owner can renounce ownership while the oracle is paused, while a manual price override is pending, or while a manual price override is already active.

After ownership is renounced, all `onlyOwner` functions become inaccessible. This means no account can call `unpause()` or `setManualPrice(0)` to clear the override state. Consequently, the oracle can remain permanently paused, or it can continue returning the manual price indefinitely once the override is active.

```
/**
 * @dev Leaves the contract without owner. It will not be possible to call
 * `onlyOwner` functions. Can only be called by the current owner.
 *
 * NOTE: Renouncing ownership will leave the contract without an owner,
 * thereby disabling any functionality that is only available to the owner.
 */
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}
```

This behavior may be intentional, but it is currently implicit rather than explicit. Because the oracle has multiple operational states controlled only by the owner, renouncing ownership makes the current state final. For comparison, `ManageableOracle`, another owner-controlled oracle wrapper, implements an explicit timelocked renounce flow through `proposeRenounceOwnership()`, `acceptRenounceOwnership()`, and `cancelRenounceOwnership()`. It also prevents accepting renounce while oracle or timelock updates are pending. `DualOracle`

instead inherits the immediate OpenZeppelin `renounceOwnership()` behavior, so the effect of renouncing while paused or while an override is pending or active is not made explicit.

Recommendations:

Make the intended ownership-renounce behaviour explicit.

If renouncing ownership is expected to be supported, document that any current pause or manual override state becomes permanent after renounce, and add tests covering renounce while paused, while an override is pending, and while an override is active.

If this behavior is not intended, override `renounceOwnership()` to either revert, or only allow renouncing when the oracle is unpaused and no manual override is pending or active:

```
manualPrice == 0 && overrideValidAt == 0
```

Customer Response:

Acknowledged.

L-03: Pausing can consume the manual-override timelock while the market is frozen

Severity: Low	Impact: Low	Likelihood: Low
Files: silo-oracles/contracts/dualOracle/DualOracle.sol	Status: Fixed	

Description:

`DualOracle` gates manual price overrides behind a timelock before the override price becomes active. This delay gives depositors and borrowers time to react before an owner-controlled price is used for solvency and LTV calculations.

However, the owner can pause the oracle while a manual override is pending. When paused, both `quote()` and `beforeQuote()` revert due to `whenNotPaused`, so market operations that depend on the oracle are blocked.

`setManualPrice()` does not have a `whenNotPaused` guard, and override activation depends only on wall-clock time:

```
overrideValidAt != 0 && overrideValidAt <= block.timestamp
```

Therefore, the owner can set a manual price, pause the oracle immediately, wait until the timelock expires, and then unpaused. Once unpaused, the manual price is active immediately. In this flow, the timelock elapsed while users could not interact with the market, so the intended reaction window can be consumed during the pause.

Recommendations:

Consider making the pause/override interaction explicit and enforce the intended timelock semantics:

- If the current behavior is intended, document that the manual-override timelock is based on wall-clock time and continues to elapse while the oracle is paused. In that case, users should not rely on the timelock as a guaranteed active-market reaction window.

- If the intended behavior is to preserve a usable reaction window, ensure paused time does not count toward override activation. This can be done by tracking when the oracle is paused while an override is pending and extending `overrideValidAt` by the paused duration on `unpause()`.
- Alternatively, the contract can disallow entering or maintaining this state combination by preventing `pause()` while an override is pending and adding `whenNotPaused` to `setManualPrice()`. This is stricter and may be less suitable if `pause()` is meant to remain available as an emergency control.

Customer Response:

Fixed in [d40504e](#) .

Fix Review:

Fixed.

Informational Severity Issues

I-01: DualOracleFactory allows nested DualOracle deployments

Files:

sil-oracles/contracts/dualOracle/DualOracleFactory.sol

Description:

`DualOracleFactory` allows a `DualOracle` to be deployed with another `DualOracle` as its underlying oracle. This can happen directly by calling `create(ISiloOracle _oracle, ...)` with an existing `DualOracle` as `_oracle`. It can also happen through the factory-deployment path, where `_underlyingOracleFactory` and `_underlyingOracleInitData` return a newly deployed `DualOracle`. For example, `_underlyingOracleFactory` can be the same `DualOracleFactory`, and `_underlyingOracleInitData` can call the direct `create(ISiloOracle, ...)` overload.

The resulting oracle is layered:

```
outer DualOracle -> inner DualOracle -> primary oracle
```

This configuration is functional, but quote behavior depends on both oracle layers. If the outer oracle is in normal mode, an inner pause causes the outer `quote()` to revert, while an active inner override determines the price returned by the outer oracle. The outer override only takes priority once it is active. This layered priority order is unintuitive, is not documented as an intended configuration, and may make the effective oracle behavior harder to reason about.

Recommendations:

If nested `DualOracle` configurations are not intended, reject same-factory `DualOracle` instances as underlying oracles in the direct `create(ISiloOracle _oracle, ...)` path. This check also mitigates the indirect factory path, because `create(address _underlyingOracleFactory, bytes _underlyingOracleInitData, ...)` eventually calls the direct `create(ISiloOracle _oracle, ...)` with the returned underlying oracle.

```
error NestedDualOracleNotAllowed();
```

```
function create(  
    ISiloOracle _oracle,
```

```
    address _owner,
    uint32 _timelock,
    uint256 _lowerPriceBound,
    uint256 _upperPriceBound,
    bytes32 _externalSalt
) public returns (IDualOracle dualOracle) {
    require(!createdInFactory[address(_oracle)], NestedDualOracleNotAllowed());

    dualOracle = _deployOracle(_externalSalt);
    DualOracle(address(dualOracle)).initialize(
        _oracle,
        _owner,
        _timelock,
        _lowerPriceBound,
        _upperPriceBound
    );
}
```

If nested **DualOracle** configurations are intended, document the layered priority order and add explicit tests for nested behaviour.

Customer Response:

Acknowledged.

I-02: No minimum timelock, or stale interface doc

Files:

sil-oracles/contracts/dualOracle/DualOracle.sol
sil-oracles/contracts/interfaces/IDualOracleFactory.sol

Description:

`initialize` validates everything except `_timelock`; the sibling `ManageableOracle` enforces 1–14 days, but those bounds were deliberately removed here. With `timelock=0`, a manual override activates the same block — no user exit window.

`IDualOracleFactory.create` natspec still claims `_timelock` "must be in `[MIN_TIMELOCK, MAX_TIMELOCK]`", constants that don't exist.

Recommendations:

Enforce a sane `_timelock` range in `initialize` (mirror `ManageableOracle`'s `MIN_TIMELOCK/MAX_TIMELOCK`), or, if instant activation is intentional, remove the `[MIN_TIMELOCK, MAX_TIMELOCK]` claim from the `IDualOracleFactory` natspec so the documented contract matches the enforced one.

Customer Response:

Fixed in [d40504e](#) .

Fix Review:

Awaiting a response

I-03: DualOracle's initialize does not verify that the wrapped oracle can quote

Files:

sil-oracles/contracts/dualOracle/DualOracle.sol

Description:

`DualOracle.initialize()` validates only part of the wrapped oracle configuration. It reads `baseToken()`, `quoteToken()`, and the base-token decimals, and requires the decimals to be non-zero. However, it does not verify that the wrapped oracle can actually price the cached base token.

This allows `DualOracle` to be deployed around an oracle that is readable but not yet quotable. In normal mode, `DualOracle.quote()` forwards directly to the wrapped oracle, so the wrapper will also revert or return no usable price. Since the wrapped oracle address is immutable after initialization, this configuration cannot be corrected by updating the primary oracle. Recovery depends on the underlying oracle later becoming functional, activating the manual override after the timelock, or deploying a new oracle configuration. For comparison, `ManageableOracle` performs a quote sanity check during oracle verification and rejects an oracle if quoting one whole base token fails or returns zero.

Recommendations:

Add a quote sanity check during `initialize()` using the cached base token and base-token decimals, so non-functional wrapped oracles are rejected during deployment:

```
error OracleQuoteFailed();
require(
    _oracle.quote(10 ** baseTokenDecimals, _baseTokenInternal) != 0,
    OracleQuoteFailed()
);
```

Customer Response:

Acknowledged – “Sometimes init of underlying oracle can happen after `DualOracle` and underlying oracle may not be ready to quote yet.”

I-04: DualOracleFactory can execute arbitrary calls during underlying oracle deployment

Files:

silo-oracles/contracts/dualOracle/DualOracleFactory.sol

Description:

`DualOracleFactory` supports a creation path that deploys the underlying oracle and then wraps it in a `DualOracle`. This path accepts an arbitrary `_underlyingOracleFactory` and arbitrary `_underlyingOracleInitData`, then performs a low-level call from the factory address:

```
(bool success, bytes memory data) =  
_underlyingOracleFactory.call(_underlyingOracleInitData);  
require(success && data.length == 32, FailedToCreateUnderlyingOracle());
```

This makes the factory able to call any target with any calldata, as long as the call returns 32 bytes that decode to an address and the returned address can be wrapped as the underlying oracle. In the current design this is mainly a trust-boundary concern, because the factory is not expected to hold funds, approvals, or privileged roles. However, if the factory address is ever granted permissions, allowlisted by another contract, or accidentally receives assets or approvals, this generic call path could allow those privileges to be used through arbitrary calldata.

Recommendations:

Document that `DualOracleFactory` must not hold assets, approvals, or privileged roles, and that `_underlyingOracleFactory` and `_underlyingOracleInitData` are trusted deployment inputs. For a stricter design, consider restricting callable factories/selectors.

Customer Response:

Acknowledged – *“Factory is treated as permissionless utility contract.”*

Design Recommendations

The following recommendations are broader design ideas for future oracle wrappers or future versions of **DualOracle**. They are not required fixes for the findings above, and may add complexity, but they can reduce operational risk when an owner-controlled oracle is used by active lending markets.

1. Split emergency and pricing authorities

DualOracle gives the same owner control over both emergency pausing and manual pricing. These actions have different risk profiles: pausing is a fast liveness control, while manual pricing directly affects solvency and liquidation outcomes.

Future oracle wrappers could separate these powers. For example, a guardian role could pause quickly during an incident, while a governance or multisig role controls manual price updates and bounds. Unpausing could also require a stronger role than pausing. This reduces the blast radius of a compromised emergency operator while keeping fast response available during oracle incidents.

2. Consider rate-limited or TWAP-style manual price movement

DualOracle applies the manual price immediately once the timelock has elapsed. This creates a discrete price transition: accounts that were healthy before activation may become liquidatable as soon as the manual override becomes active.

For future designs, Silo could consider applying manual override prices gradually. For example, the oracle could enforce a configurable maximum price change over a given time window, or move from the last valid price toward the manual price using a TWAP-style transition. This can reduce abrupt liquidation effects from owner-controlled repricing. The tradeoff is that emergency correction becomes slower, so the rate limit should be configurable per market or paired with a clearly defined emergency process.

3. Make the return to primary pricing explicit

`DualOracle` applies a timelock before entering manual override mode, but exiting manual mode is immediate. Calling `setManualPrice(0)` clears `manualPrice` and `overrideValidAt`, so the next `quote()` is served by the primary oracle in the same transaction context.

This is simple, but it makes the return path depend entirely on operator timing. If the primary oracle has recovered at a materially different price, the market can experience a second abrupt repricing event. If the primary oracle has not actually recovered, clearing the override can move the market back to a reverting oracle, and re-enabling the manual price then requires waiting through the full timelock again.

A future version could check that the primary oracle returns a valid non-zero quote before clearing an active manual override. The return to primary pricing could also use a configurable timelock, giving users a reaction window before the oracle switches away from the manual price. If immediate clearing is preferred for operational reasons, this behavior should be documented as an intentional tradeoff.

4. Consider an emergency price-limit mode

`DualOracle` currently supports a manual price override where, once active, `quote()` stops using the wrapped oracle and returns the owner-provided `manualPrice`. This is useful when the primary oracle is unavailable or clearly unsafe.

A future version could also support a price-limit mode for cases where the primary oracle still returns prices but may move outside an acceptable range. In this mode, the oracle would continue reading from the primary oracle. If the primary quote is within the configured lower and upper limits, it is returned as-is. If it falls below the lower limit, the lower limit is returned. If it rises above the upper limit, the upper limit is returned.

This gives operators a less absolute emergency tool than fully replacing the primary oracle. The primary oracle still drives pricing while it remains within the accepted range, and the owner-controlled limits only constrain extreme moves. The tradeoff is that this mode does not help when the primary oracle reverts completely, so it should complement rather than replace pause and manual override behavior.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.